

Single Layer Perceptron to Multi Layer Perceptron

Afiahayati
Magister Kecerdasan Artifisial
Departemen Ilmu Komputer dan Elektronika
FMIPA Universitas Gadjah Mada
afia@ugm.ac.id



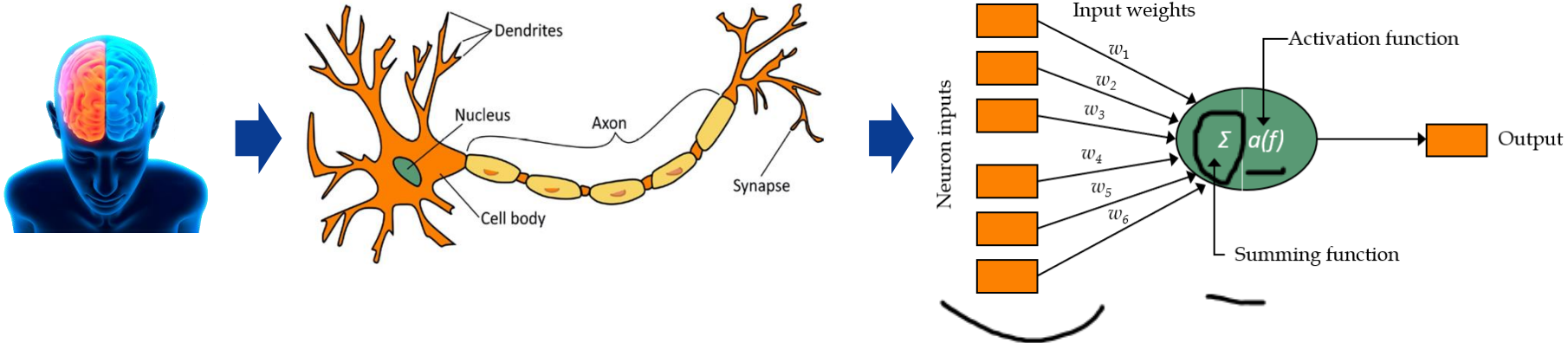
Agenda (Update)

- Meeting 1 : Perkenalan, RPKPS, Linear Classifier (24 Aug) – offline
- Meeting 2 : ANN – SLP and MLP – **submit Assignment 1 (7 Sept) – offline**
- Meeting 3 : Discussion Assignment 1 - **submit Assignment 2 (14 Sept) – offline**
- Meeting 4 : Guest Lecturer (19 Sept) – online
- Meeting 5 : Learning Strategy + Discussion Assignment 1 (21 Sept) – offline
- Meeting 6 : Introduction to Deep Learning + CNN (28 Sept) - offline
- Meeting 7 : Discussion and Presentation - **Mini Project** - (16 Oct) - online
- **Midterm Examination**

Artificial Neural Network (ANN)

- Salah satu metode mesin pembelajaran yang terinspirasi oleh cara kerja jaringan saraf biologis di otak manusia
- Merupakan jaringan dari unit pemroses kecil yang saling terhubung, yang dimodelkan berdasar sistem saraf manusia
- Konsep ANN bermula pada artikel dari Waffen McCulloch dan Walter Pitts pada tahun 1943 yaitu mencoba untuk memformulasikan model matematis sel-sel otak manusia

Jaringan saraf biologis	ANN
Soma	Neuron
Dendrite	Input
Axon	Output
Synapse	Weight



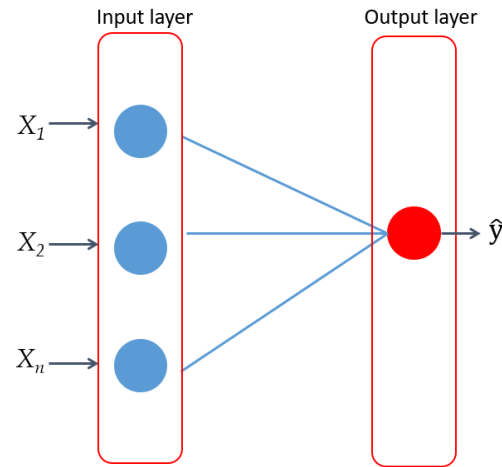
Arsitektur Single-layer Perceptron

Arsitektur single-layer ANN hanya terdiri dari input layer dan output layer

Unit pemrosesan informasi pada ANN sebagai berikut:

- Satu set link berupa neuron dan bobot w
- Fungsi penambah (penggabung linear) untuk menghitung jumlah perkalian bobot terhadap input X
- Fungsi aktivasi $a(\cdot)$

$$y = a(f)$$



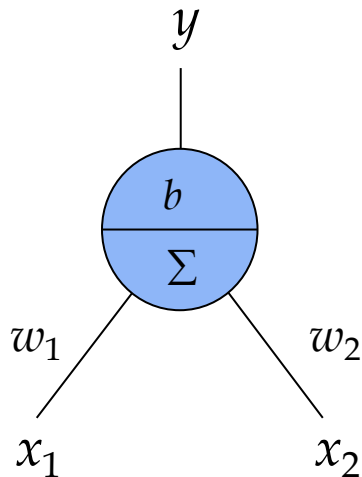
$$f = \sum_{i=1}^m \underbrace{w_i x_i}_{\text{weight} \times \text{input}} + \underbrace{b}_{\text{bias}}$$

$y = a(f)$

Apa yang bisa dilakukan sebuah Neuron ?

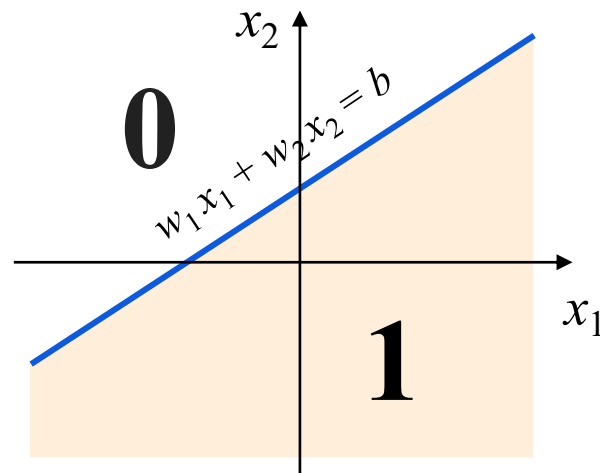
Sebuah neuron pada ANN dapat menyelesaikan permasalahan klasifikasi biner

- Sebagai fungsi pemisah (*hyperspace separation*)
- Sebagai *binary threshold*



$$f(x) = w_1x_1 + w_2x_2 - b$$

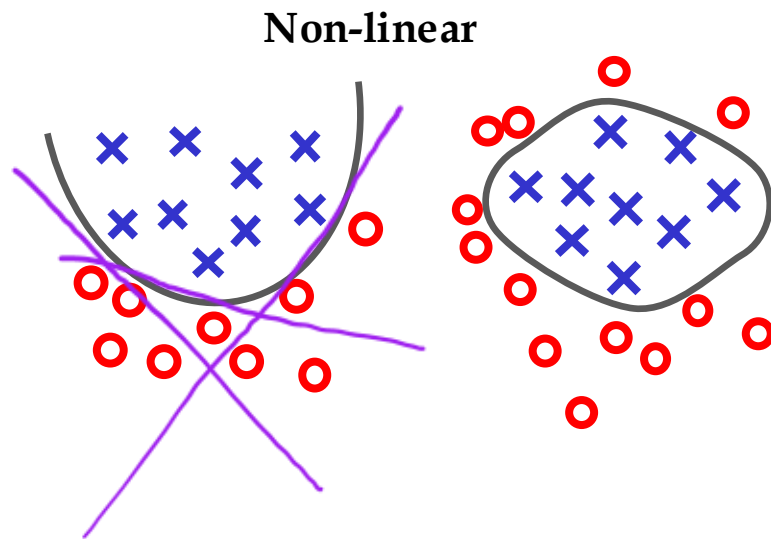
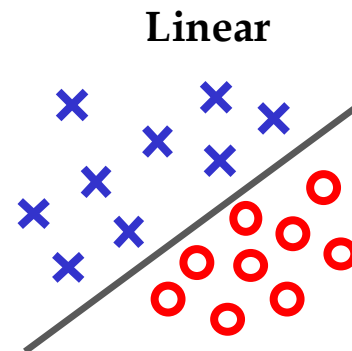
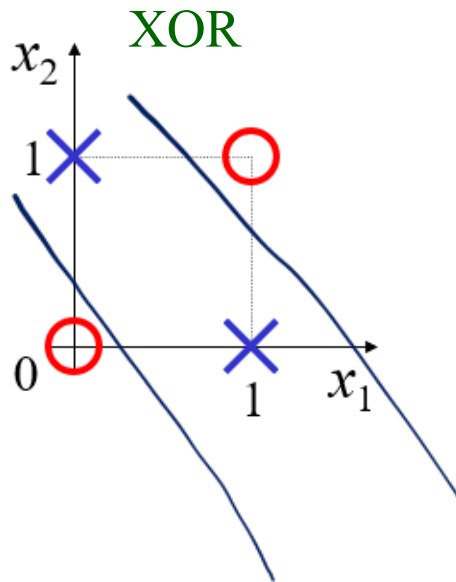
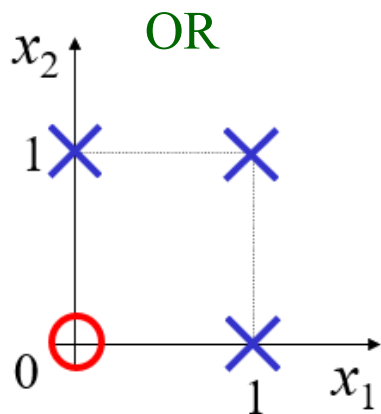
$$y = \begin{cases} 1 & f(x) \geq 0 \\ 0 & \text{otherwise} \end{cases}$$



Permasalahan Linear dan Non-Linear

Permasalahan klasifikasi dapat dikategorikan sebagai:

- Permasalahan Linear, misalnya fungsi OR dan AND
- Permasalahan Non-Linear, misalnya fungsi XOR

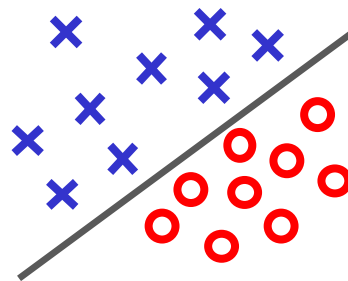


Permasalahan Linear dan Non-Linear

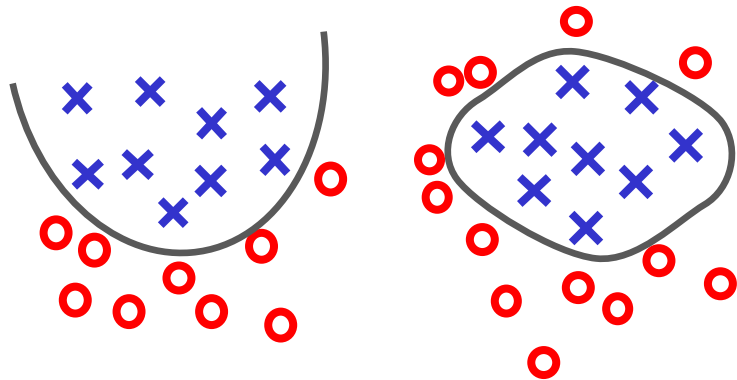
Permasalahan klasifikasi dapat dikategorikan sebagai:

- Permasalahan Linear, misalnya fungsi OR dan AND
- Permasalahan Non-Linear, misalnya fungsi XOR

Linear



Non-linear

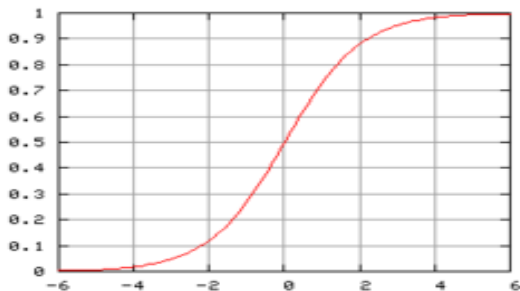


Fungsi Aktivasi

- Fungsi aktivasi merubah neuron menjadi non-linear
- Beberapa contoh fungsi aktivasi yang umum digunakan pada metode ANN sebagai berikut

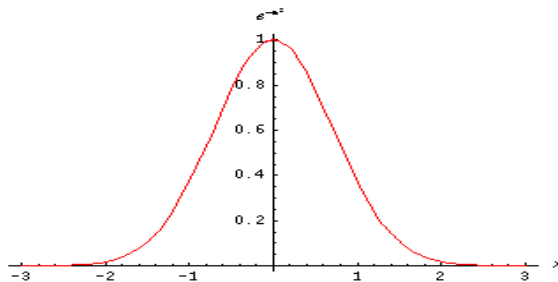
Sigmoid function

$$a(f) = \frac{1}{1 + \exp(-f)}$$



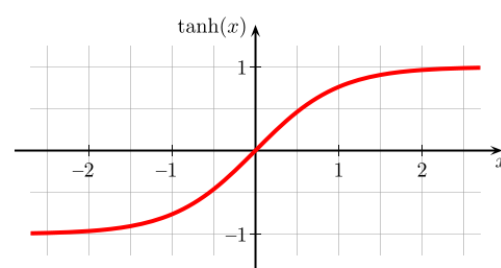
Gaussian function

$$a(f) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{1}{2}\left(\frac{f-\mu}{\sigma}\right)^2\right)$$

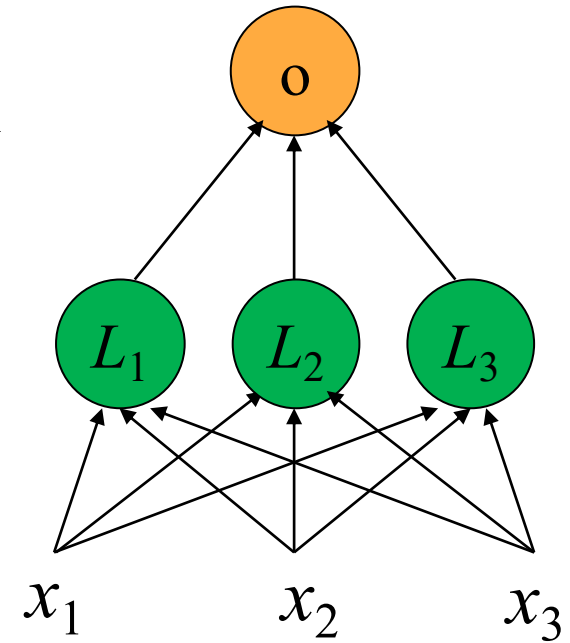
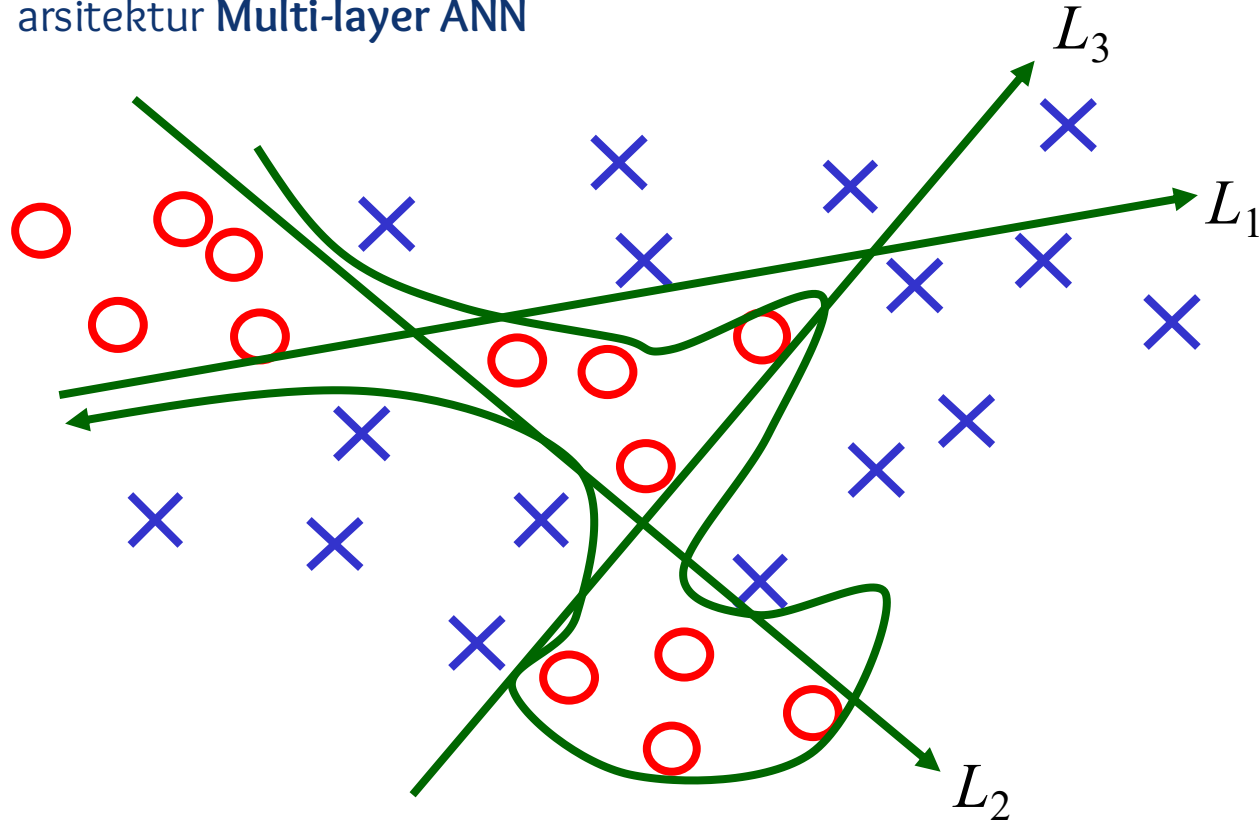


Tangent Hyperbolic function

$$\tanh(x) = 2\sigma(2x) - 1$$

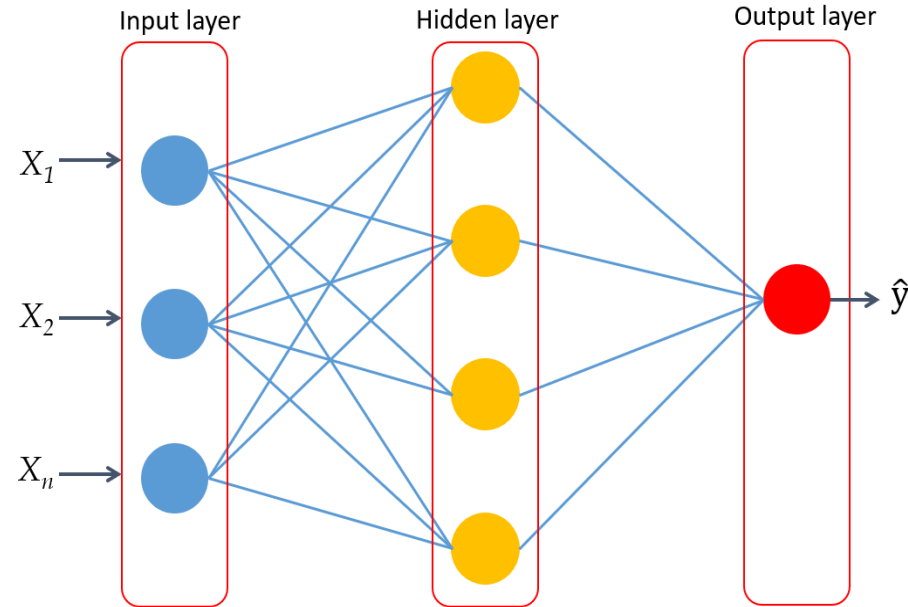


Pada permasalahan non-linear dan permasalahan yang lebih kompleks, menggunakan arsitektur **Multi-layer ANN**



Arsitektur Multi-layer ANN

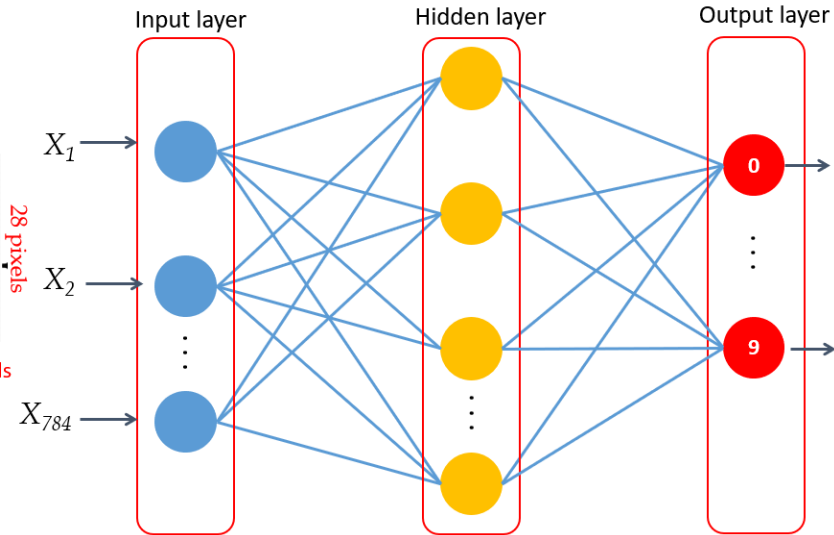
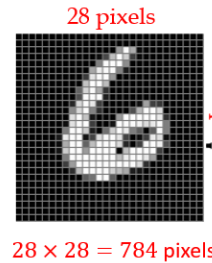
- Terdiri dari tiga layer yaitu:
 - input layer
 - hidden layer
 - output layer
- Hubungan antar neuron pada ANN merupakan **fully connected network (FCN)**
- **Jumlah hidden layer** sebaiknya disesuaikan dengan kompleksitas permasalahan
- **Jumlah neuron pada hidden layer** umumnya lebih banyak daripada jumlah neuron di *output layer*



Desain arsitektur ANN

Penentuan jumlah *neuron* pada *input layer*

- Jumlah neuron sesuai dengan jumlah fitur pada data input



Penentuan jumlah *neuron* pada *output layer*

- Jumlah neuron sesuai dengan permasalahan
- Pada permasalahan klasifikasi biner dan regresi bisa menggunakan hanya satu *neuron*
- Pada permasalahan klasifikasi *multiclass* menggunakan jumlah *neuron* sesuai jumlah label kelasnya, misalnya: 10 neuron pada pengenalan angka

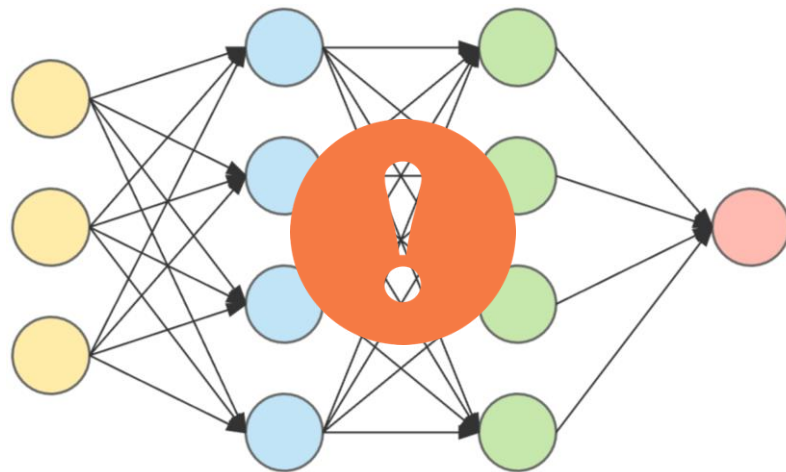
Desain arsitektur ANN

Penentuan jumlah *hidden layer*:

- Semakin banyak jumlah layer memerlukan komputasi waktu lebih lama
- Jumlah layer sebaiknya disesuaikan dengan kompleksitas permasalahan

Penentuan jumlah node (neuron) pada *hidden layer*:

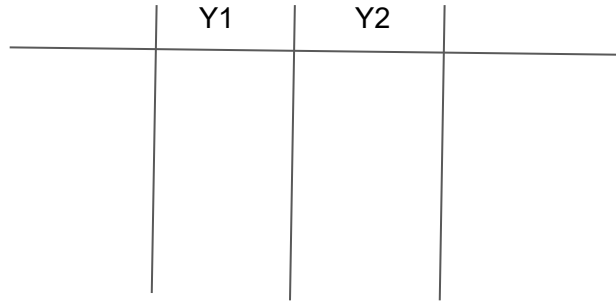
- Semakin banyak jumlah node memungkinkan mempelajari pola yang lebih rumit
- Untuk mencegah *overfitting* sebaiknya menambah jumlah node secara bertahap



Output Layer in MLP

- Binary Representation
- Each Class Representation (One hot encoding)

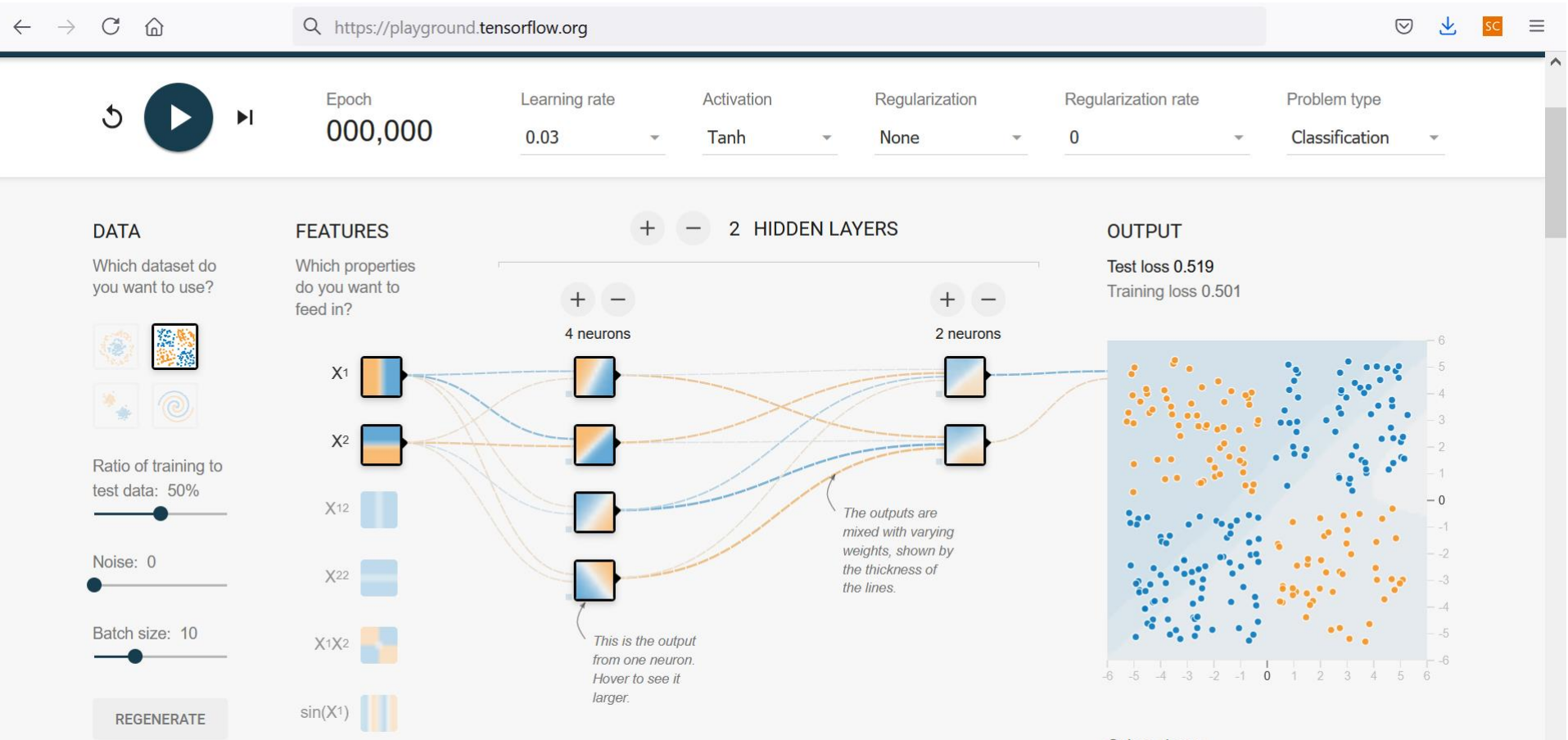
5 class in binary representation
=> 3 neurons in output layer



5 class in binary representation
=> 5 neurons in output layer

Desain arsitektur ANN

<https://playground.tensorflow.org/>



Tahapan ANN: *Feed Forward*

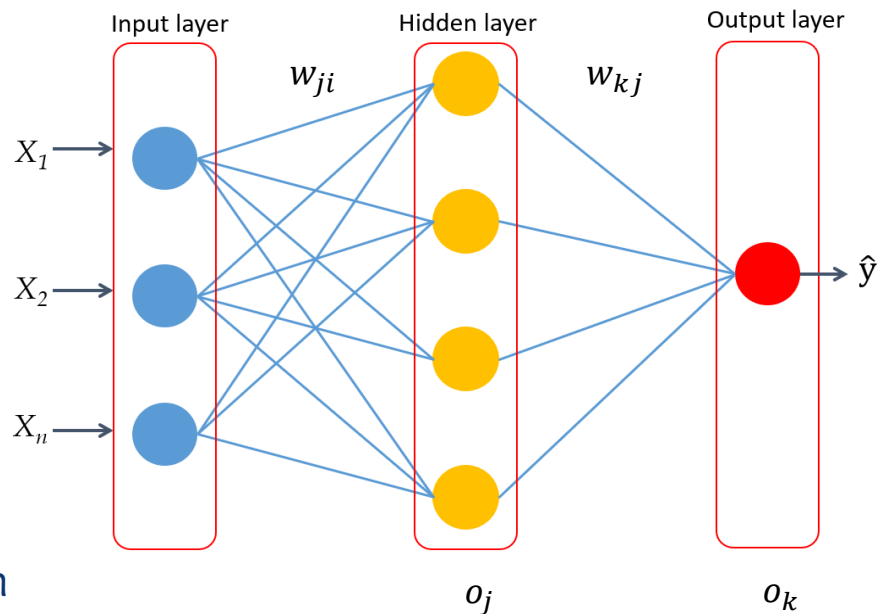
1. Masukkan vektor X ke *input layer*
2. Hitung output setiap neuron di *hidden layer* dan *output layer*

$$o_j = \sum_{i=1}^n w_{ji}x_i + b \quad o_k = \sum_{j=1}^m w_{kj}o_j + b$$

dimana x_i dan o_j adalah matrik input dan output neuron pada layer sebelumnya

w_{ji} dan w_{kj} adalah bobot yang menghubungkan antara neuron pada dua layer berbeda, dan b adalah bias
 n dan m adalah jumlah neuron di layer sebelumnya

3. Hitung fungsi aktivasi pada layer output $\hat{y} = a(o_k)$



Tahapan ANN: Pembelajaran (*Learning*)

1. Inisialisasi bobot W
2. Update bobot sehingga output dari ANN adalah konsisten dengan label kelas pada data latih

a. Fungsi obyektif (fungsi loss): $J(W) = \frac{1}{n} \sum_{i=1}^n \left(y^{(i)} - f(x^{(i)}; W) \right)^2$

y – target /gt
 f – prediksi

- b. Menemukan bobot baru dengan meminimalkan fungsi obyektif, contoh:
algoritma backpropagation

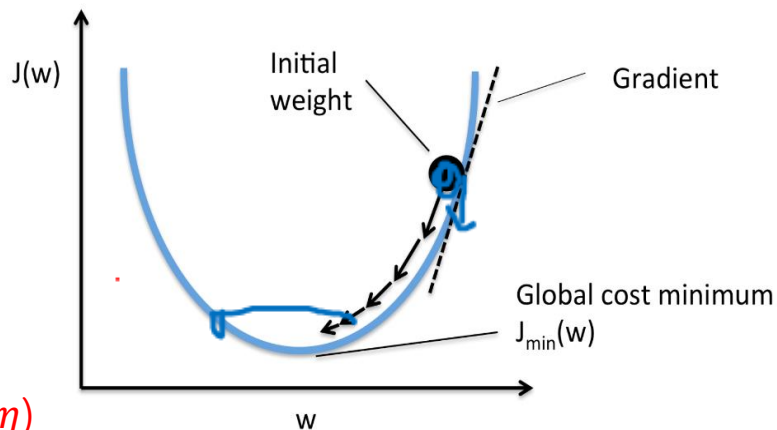
Algoritma Pembelajaran (Learning)

Untuk merancang algoritma pembelajaran, ada beberapa hal yang perlu diperhatikan:

1. Kriteria Iterasi berhenti? **konvergen, iterasi (epoch)**
2. Bagaimana direction? **gradient descent**
3. Berapa banyak (step) yang diperlukan? **nilai learning rate (η)**

Algoritma Backpropagation Gradient Descent

- Inisialisasi bobot secara random
- Melakukan iterasi sampai konvergen atau maksimum iterasi
 - Hitung gradient, $\frac{\partial J(W)}{\partial W}$
 - Update bobot, $W \leftarrow W - \eta \frac{\partial J(W)}{\partial W}$
- Mengembalikan nilai bobot



$$W^* = \underset{W}{\operatorname{argmin}} J(W)$$

$$J(W) = \frac{1}{n} \sum_{i=1}^n \left(y^{(i)} - f(x^{(i)}; W) \right)^2$$

Nilai *learning rate* jika terlalu kecil memerlukan waktu lebih lama untuk konvergen, jika terlalu besar membuat model tidak stabil

Algoritma Pembelajaran (*Learning*)

Algoritma Stochastic Gradient Descent

- Inisialisasi bobot secara random
- Melakukan iterasi sampai konvergen atau maksimum iterasi
 - Baca **setiap** data poin i
 - Hitung gradient, $\frac{\partial J_i(W)}{\partial W}$
 - Update bobot, $W \leftarrow W - \eta \frac{\partial J(W)}{\partial W}$
- Mengembalikan nilai bobot

- Inisialisasi bobot secara random
- Melakukan iterasi sampai konvergen atau maksimum iterasi
 - Baca **batch B** data poin
 - Hitung gradient, $\frac{\partial J(W)}{\partial W} = \frac{1}{B} \sum_{k=1}^B \frac{\partial J_k(W)}{\partial W}$
 - Update bobot, $W \leftarrow W - \eta \frac{\partial J(W)}{\partial W}$
- Mengembalikan nilai bobot

Tahapan Pembelajaran Multi-layer Perceptron ANN

Langkah 0 – Inisialisasi bobot, learning rate, maksimum iterasi

Langkah 1 – Membaca vektor input X

Langkah 2 – Lakukan iterasi (*epoch*)

Langkah 3 – Hitung luaran neuron di hidden layer dan output layer ✓

Langkah 4 – Hitung *back propagate error* (pada output layer dan hidden layer) .

Langkah 5 – Perbarui semua bobot (pada output layer dan hidden layer) →

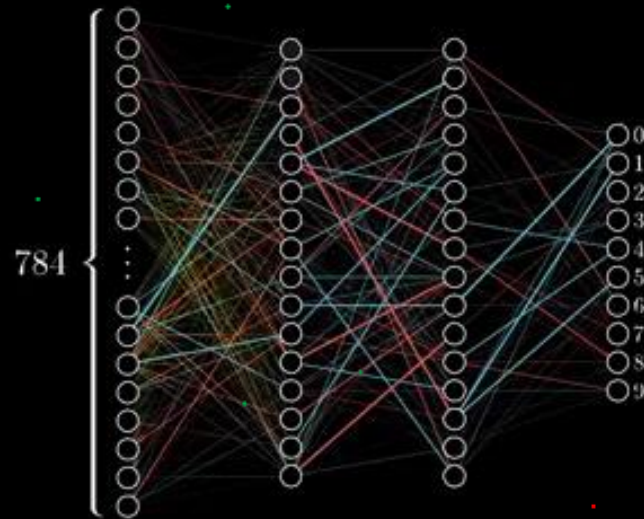
Langkah 6 – Ulangi langkah 3 – 5 hingga bobot konvergen atau maksimum iterasi

Langkah 7 – Luaran berupa matrik bobot (pada output layer dan hidden layer)

<https://machinelearningmastery.com/implement-backpropagation-algorithm-scratch-python/>

Contoh Proses Pembelajaran ANN

Training in
progress...



Tahapan implementasi ANN

- **Load data**: membaca file data input
- **Split data**: membagi data menjadi data latih, data validasi, data uji
- **Define model**: merancang arsitektur atau model ANN
- **Compile model**: menjalankan model ANN yang sudah dirancang
- **Fit model**: membangun model ANN berdasarkan data latih
- **Evaluation model**: mengevaluasi model ANN berdasarkan data validasi
- **Save model**: menyimpan model ANN
- **Prediction**: memprediksi output dari data uji menggunakan model ANN yang terbaik

Bisa digabung
menjadi satu

Tools / Lab Online

Scikit-learn

sklearn.neural_network.MLPClassifier

```
class sklearn.neural_network.MLPClassifier(hidden_layer_sizes=100, activation='relu', *, solver='adam', alpha=0.0001,
batch_size='auto', learning_rate='constant', learning_rate_init=0.001, power_t=0.5, max_iter=200, shuffle=True,
random_state=None, tol=0.0001, verbose=False, warm_start=False, momentum=0.9, nesterovs_momentum=True,
early_stopping=False, validation_fraction=0.1, beta_1=0.9, beta_2=0.999, epsilon=1e-08, n_iter_no_change=10, max_fun=15000)
```

[\[source\]](#)

Parameter MLPClassifier:

- hidden_layer size: jumlah neuron di *hidden layer*
- activation: fungsi aktivasi yang digunakan di *hidden layer*
- solver: metode adaptive learning rate yang digunakan
- batch_size: ukuran batch
- learning_rate_init: inisialisasi *learning rate*
- max_iter: *maksimum iterasi*
- early_stopping: bernilai false jika tidak menerapkan *early stopping*

Contoh implementasi ANN menggunakan library Scikit-learn

- Load data

`sklearn.datasets.load_iris`

```
sklearn.datasets.load_iris(*, return_X_y=False, as_frame=False)
```

Load and return the iris dataset (classification).

The iris dataset is a classic and very easy multi-class classification dataset.

Classes	3
Samples per class	50
Samples total	150
Dimensionality	4
Features	real, positive

```
from sklearn import datasets  
iris = datasets.load_iris()  
X = iris.data  
y = iris.target
```

Samples
(instances, observations)

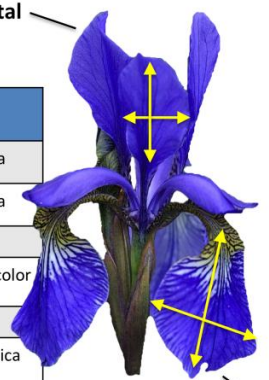
	Sepal length	Sepal width	Petal length	Petal width	Class label
1	5.1	3.5	1.4	0.2	Setosa
2	4.9	3.0	1.4	0.2	Setosa
...					
50	6.4	3.5	4.5	1.2	Versicolor
...					
150	5.9	3.0	5.0	1.8	Virginica

Features
(attributes, measurements, dimensions)

Class labels
(targets)

Petal

Sepal



Contoh implementasi ANN menggunakan library Scikit-learn

- Split data https://scikit-learn.org/stable/modules/generated/sklearn.neural_network.MLPClassifier.html

`sklearn.model_selection.train_test_split`

```
sklearn.model_selection.train_test_split(*arrays, test_size=None, train_size=None, random_state=None, shuffle=True, stratify=None)
```

[source]

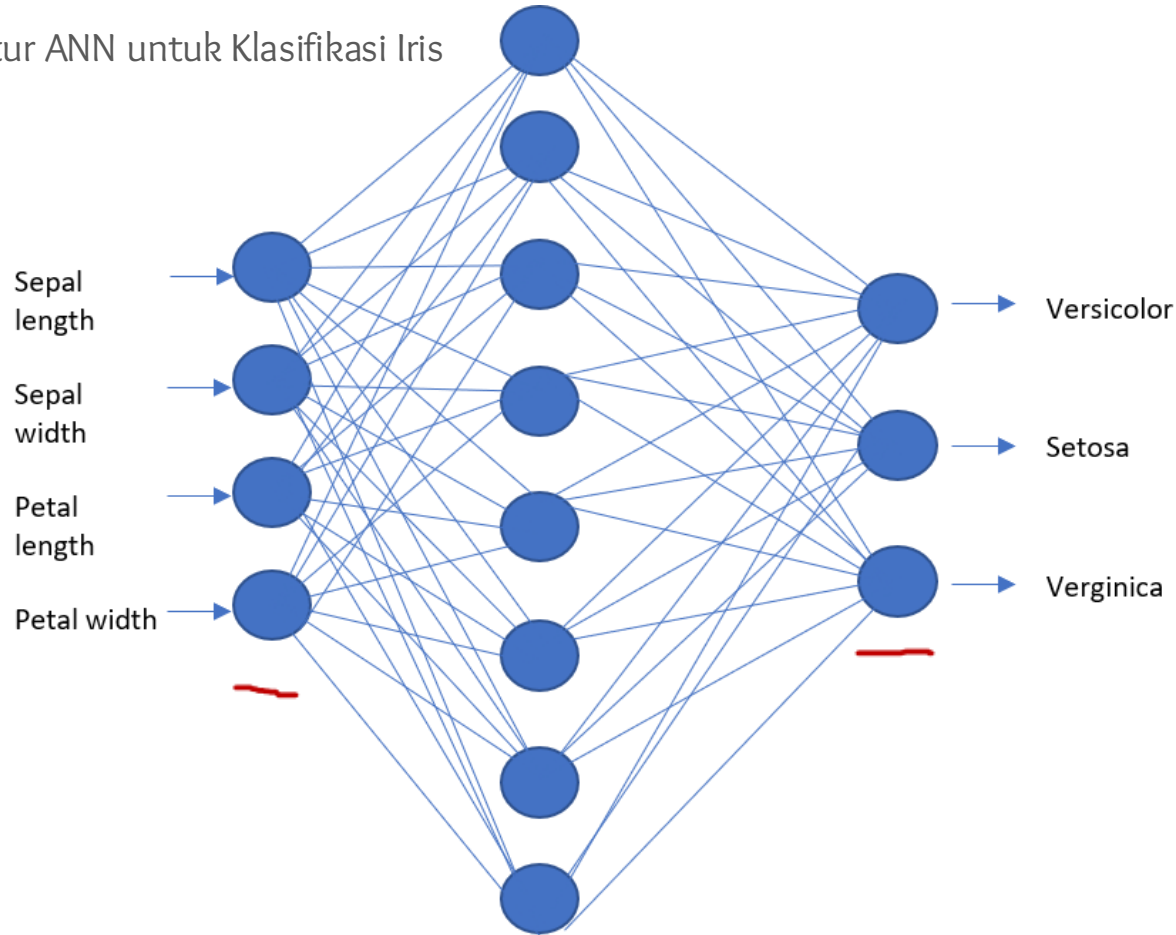
```
from sklearn.model_selection import train_test_split

X_train, X_test, Y_train, Y_test = train_test_split(X, y, test_size=.10)
X_train, X_val, Y_train, Y_val = train_test_split(X_train, Y_train, test_size=.2)
print('X_train', X_train.shape)
print('X_val', X_val.shape)
print('X_test', X_test.shape)
```

X_train	(108, 4)
X_val	(27, 4)
X_test	(15, 4)

Contoh implementasi ANN menggunakan library Scikit-learn

Arsitektur ANN untuk Klasifikasi Iris



Iris Versicolor



Iris Setosa



Iris Virginica

Contoh implementasi ANN menggunakan library Scikit-learn

- Define and compile model

```
from sklearn.neural_network import MLPClassifier
```

```
mlp = MLPClassifier(hidden_layer_sizes=(100, ), activation='logistic', max_iter= 800)
```

- Fit model and evaluation model

```
from sklearn.metrics import accuracy_score
```

```
mlp.fit(X_train, Y_train)
```

```
prediksi_val = mlp.predict(X_val)
```

```
acc_val = accuracy_score(Y_val, prediksi_val)
```

```
print('Akurasi Validasi Training ANN:', acc_val)
```

Akurasi Validasi Training ANN: 1.0

Contoh implementasi ANN menggunakan library Scikit-learn

- Prediction

```
from sklearn.metrics import accuracy_score, plot_confusion_matrix
```

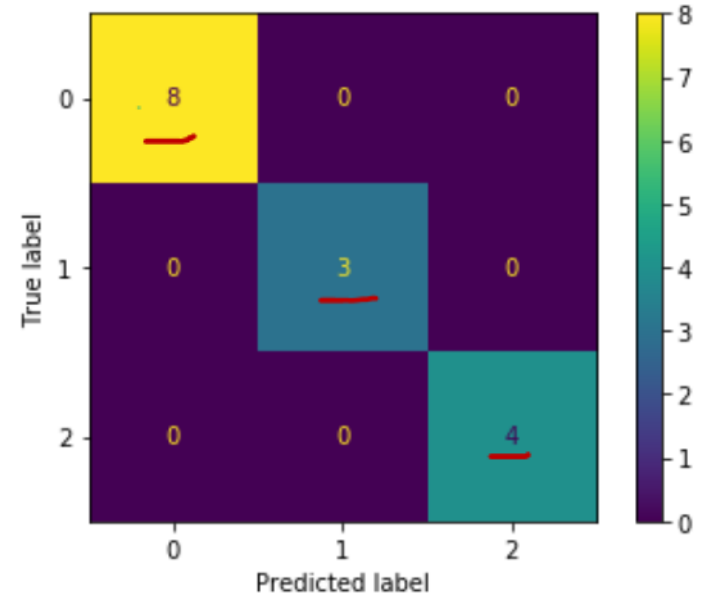
```
prediksi_test = mlp.predict(X_test)
```

```
acc_test = accuracy_score(Y_test, prediksi_test)
```

```
print('Akurasi Testing ANN:', acc_test)
```

```
plot_confusion_matrix(mlp, X_test, Y_test)
```

Akurasi Testing ANN: 1.0



Tools / Lab Online

- TensorFlow is an end-to-end open-source platform for machine learning
- Keras is a deep learning API written in Python, running on top of the machine learning platform TensorFlow. It was developed with a focus on enabling fast experimentation. Being able to go from idea to result as fast as possible is key to doing good research.



<https://www.tensorflow.org/overview/>

https://keras.io/getting_started/

<https://keras.io/examples/>

Tools / Lab Online

- Keras Model Sequential dan Layers

<https://keras.io/api/models/>

https://keras.io/guides/sequential_model/

<https://keras.io/api/layers/>

```
model = keras.Sequential(  
    [  
        layers.Dense(64, activation="relu", name="layer1"),  
        layers.Dense(32, activation="relu", name="layer2"),  
        layers.Dense(4, name="layer3"),  
    ]  
)
```

atau

```
model = keras.Sequential()  
model.add(layers.Dense(64, activation="relu"))  
model.add(layers.Dense(32, activation="relu"))  
model.add(layers.Dense(4))
```

Contoh implementasi ANN menggunakan library keras

- Load data

```
from sklearn import datasets
from sklearn.model_selection import train_test_split
from keras.utils import to_categorical

iris = datasets.load_iris()
X = iris.data
y = iris.target

X_train, X_test, Y_train, Y_test = train_test_split(X, y, test_size=.10)
X_train, X_val, Y_train, Y_val = train_test_split(X_train, Y_train, test_size=.2)
print('X_train', X_train.shape)
print('X_val', X_val.shape)
print('X_test', X_test.shape)

Y_train = to_categorical(Y_train,3)
Y_val = to_categorical(Y_val,3)
Y_test = to_categorical(Y_test,3)
```

Contoh implementasi ANN menggunakan library keras

- Define model dan compile model

```
from keras.models import Sequential
from keras.layers import Flatten, Dense

model = Sequential()
model.add(Flatten())
model.add(Dense(64,activation='relu'))
model.add(Dense(3,activation='softmax'))

model.compile(optimizer='adam',loss='categorical_crossentropy', metrics=['acc'])
```

Contoh implementasi ANN menggunakan library keras

- Fit model

```
model.fit(X_train,Y_train,epochs=64,batch_size=5,validation_data=(X_val,Y_val))  
model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
flatten (Flatten)	(None, 4)	0
dense (Dense)	(None, 64)	320
dense_1 (Dense)	(None, 3)	195

Total params: 515

Trainable params: 515

Non-trainable params: 0

Contoh implementasi ANN menggunakan library keras

- Evaluation model

```
loss, accuracy = model.evaluate(X_test, Y_test)
print('Akurasi Testing MLP:', accuracy)
```

1/1 [=====] - 0s 0s/step - loss: 0.0393 - acc: 1.0000
Akurasi Testing ANN: 1.0



Contoh implementasi arsitektur *fully connected layer* pada pengenalan angka

Dataset MNIST *Handwritten Digit* dibagi menjadi 3:

- 55,000 training data ↴
- 10,000 test data ↴
- 5,000 validation data ↴

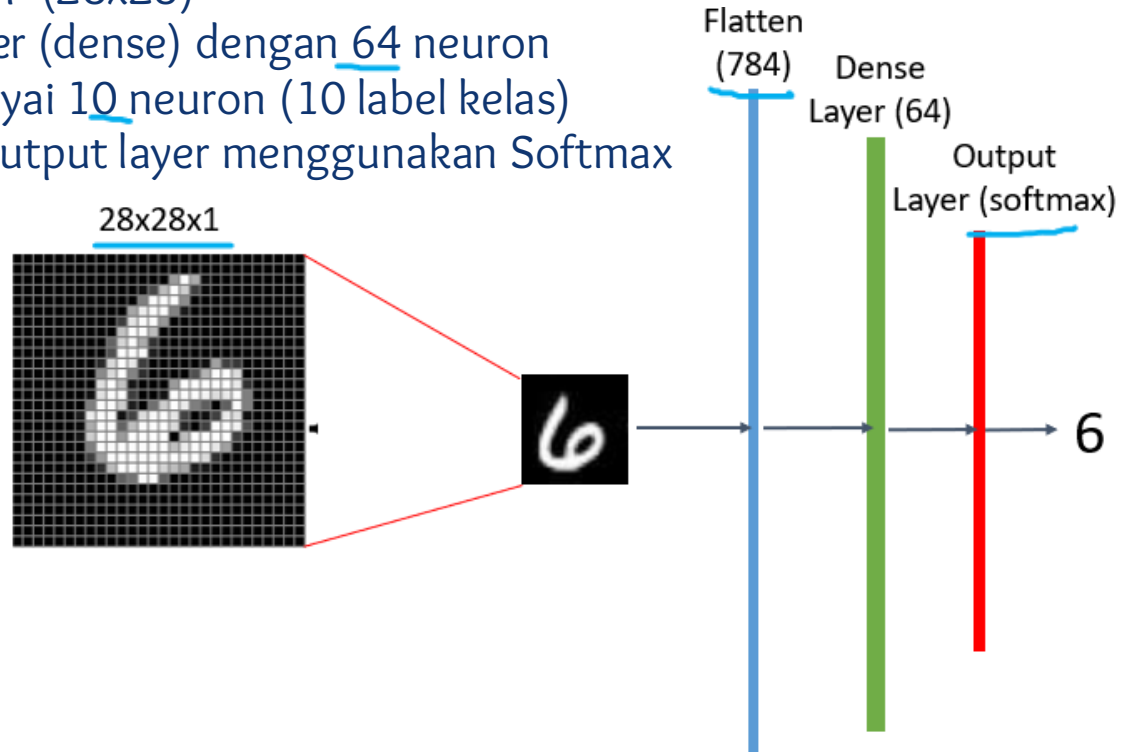
Setiap citra berukuran 28 × 28 pixels dan label kelas dirubah menjadi *one hot encoded*

label = 5 	label = 0 	label = 4 	label = 1 	label = 9 	0	[1 0 0 0 0 0 0 0 0 0]
label = 2 	label = 1 	label = 3 	label = 1 	label = 4 	1	[0 1 0 0 0 0 0 0 0 0]
label = 3 	label = 5 	label = 3 	label = 6 	label = 1 	2	[0 0 1 0 0 0 0 0 0 0]
label = 3 	label = 5 	label = 3 	label = 6 	label = 1 	3	[0 0 0 1 0 0 0 0 0 0]
label = 3 	label = 5 	label = 3 	label = 6 	label = 1 	4	[0 0 0 0 1 0 0 0 0 0]
label = 3 	label = 5 	label = 3 	label = 6 	label = 1 	5	[0 0 0 0 0 1 0 0 0 0]
label = 3 	label = 5 	label = 3 	label = 6 	label = 1 	6	[0 0 0 0 0 0 1 0 0 0]
label = 3 	label = 5 	label = 3 	label = 6 	label = 1 	7	[0 0 0 0 0 0 0 1 0 0]

Contoh implementasi arsitektur *fully connected layer* pada pengenalan angka

Arsitektur ANN yang digunakan untuk pengenalan angka:

- Ukuran input layer 784 (28x28)
- Terdapat 1 hidden layer (dense) dengan 64 neuron
- Output layer mempunyai 10 neuron (10 label kelas)
- Fungsi aktivasi pada output layer menggunakan Softmax



Contoh implementasi arsitektur *fully connected layer* pada pengenalan angka

1. Load data

```
import keras
from keras.datasets import mnist

(X_train, y_train), (X_test, y_test) = mnist.load_data()
X_train = X_train.reshape(-1, 28,28,1)
X_test = X_test.reshape(-1, 28,28,1)
X_train = X_train.astype('float32')
X_test = X_test.astype('float32')
X_train /= 255
X_test /= 255
y_train = keras.utils.to_categorical(y_train, 10)
y_test = keras.utils.to_categorical(y_test, 10)
```

Contoh implementasi arsitektur *fully connected layer* pada pengenalan angka

2. Define Model

```
from keras.models import Sequential
from keras.layers import Flatten, Dense

model1 = Sequential()
model1.add(Flatten())
model1.add(Dense(64,activation='relu'))
model1.add(Dense(10,activation='softmax'))
```

Model: "sequential"

Layer (type)	Output Shape	Param #
flatten (Flatten)	(None, 784)	0
dense (Dense)	(None, 64)	50240
dense_1 (Dense)	(None, 10)	650

Total params: 50,890

Trainable params: 50,890

Non-trainable params: 0

Contoh implementasi arsitektur *fully connected layer* pada pengenalan angka

3. Compile Model, Fit Model, Save Model, dan Evaluasi Model

```
model1.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['acc'])  
history =  
model1.fit(X_train, y_train, epochs=10, batch_size=100, validation_data=(X_test, y_test))  
model1.save('my_model1.h5')  
model1.evaluate(X_test, y_test)
```

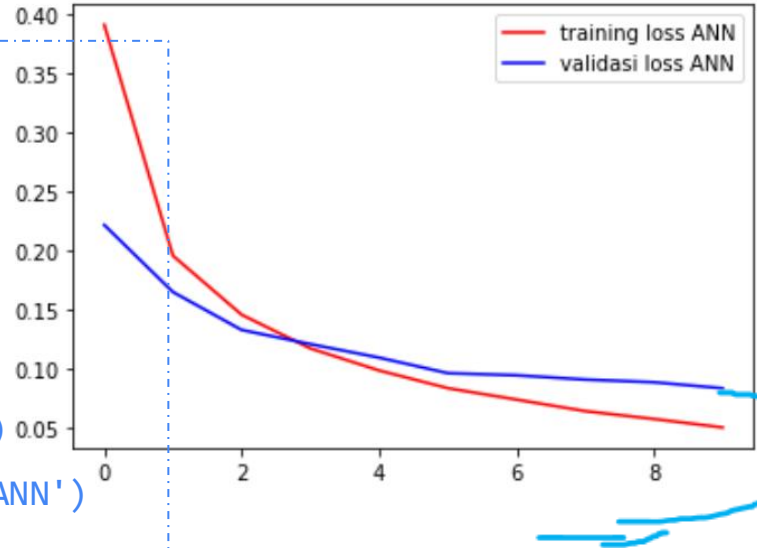
```
313/313 [=====] - 0s 1ms/step - loss: 0.0839 - acc: 0.9761  
[0.08388985693454742, 0.9761000275611877]
```

Contoh implementasi arsitektur *fully connected layer* pada pengenalan angka

3. Visualiasasi Evaluasi Model

```
import matplotlib.pyplot as plt

epochs = range(10)
loss1 = history1.history['loss']
val_loss1 = history1.history['val_loss']
plt.plot(epochs,loss1,'r',label='training loss ANN')
plt.plot(epochs,val_loss1,'b',label='validasi loss ANN')
plt.legend()
```



Contoh implementasi arsitektur *fully connected layer* pada pengenalan angka

3. Load Model dan Prediction

```
import numpy as np
from keras.models import load_model

model_simpan = load_model('my_model.h5')
pred = model_simpan.predict(X_test)
print('label actual:', np.argmax(y_test[30]))
print('label prediction:', np.argmax(pred[30]))
```

```
label actual: 3
label prediction: 3
```